

IQMS — Intelligent Queue Management System

Complete Technical Documentation
April 2026

This document consolidates the content of `DEVELOPER_DOC.md`, `SIMULATOR_OVERVIEW.md`, and `BUG_FIXES.md` into a single reference.

Table of Contents

- System Overview
 - Repository Layout
 - Architecture & Data Flow
 - Pipeline A — Head-Detector
 - Pipeline B — Queue-Management-System-v2
 - Forecasting Layer
 - Database Schema
 - Configuration Reference
 - Grafana / TimescaleDB Integration
 - Setup & First Run
 - Simulator
 - Bug Fixes & Data Quality History
 - Known Limitations
-

1. System Overview

IQMS monitors a retail entrance zone via RTSP camera feeds, detects and tracks people, stores per-person events in PostgreSQL/TimescaleDB, and forecasts queue load for the next 60 minutes using an ensemble of Prophet + LSTM + XGBoost. A Grafana dashboard consumes the live and forecast tables for real-time operations.

Two parallel detection pipelines exist. They write to the same database tables and are interchangeable depending on hardware:

Pipeline	Model	Extra data	Use when
Head-Detector (A)	YOLOv9 (head pose)	head direction	GPU / TensorRT available
Queue-Management-v2 (B)	YOLOv9 body + Uniface	gender, age, bag detection	CPU or when face analytics needed

Both pipelines:

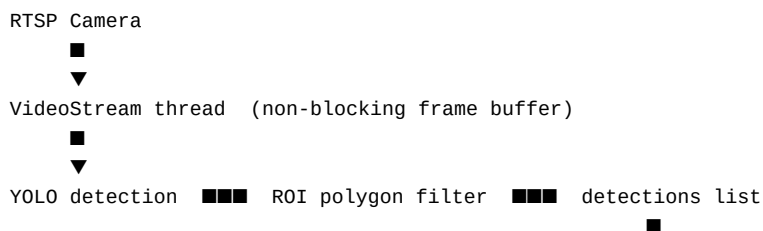
- Use the **insert-at-death** pattern: one DB row is written per confirmed track when the track disappears, not on first detection
- Write periodic **queue-state snapshots** every 10 seconds to `queue_state_snapshots`
- Tag test-video runs as `SIM_live_test_video` to exclude them from real-data queries

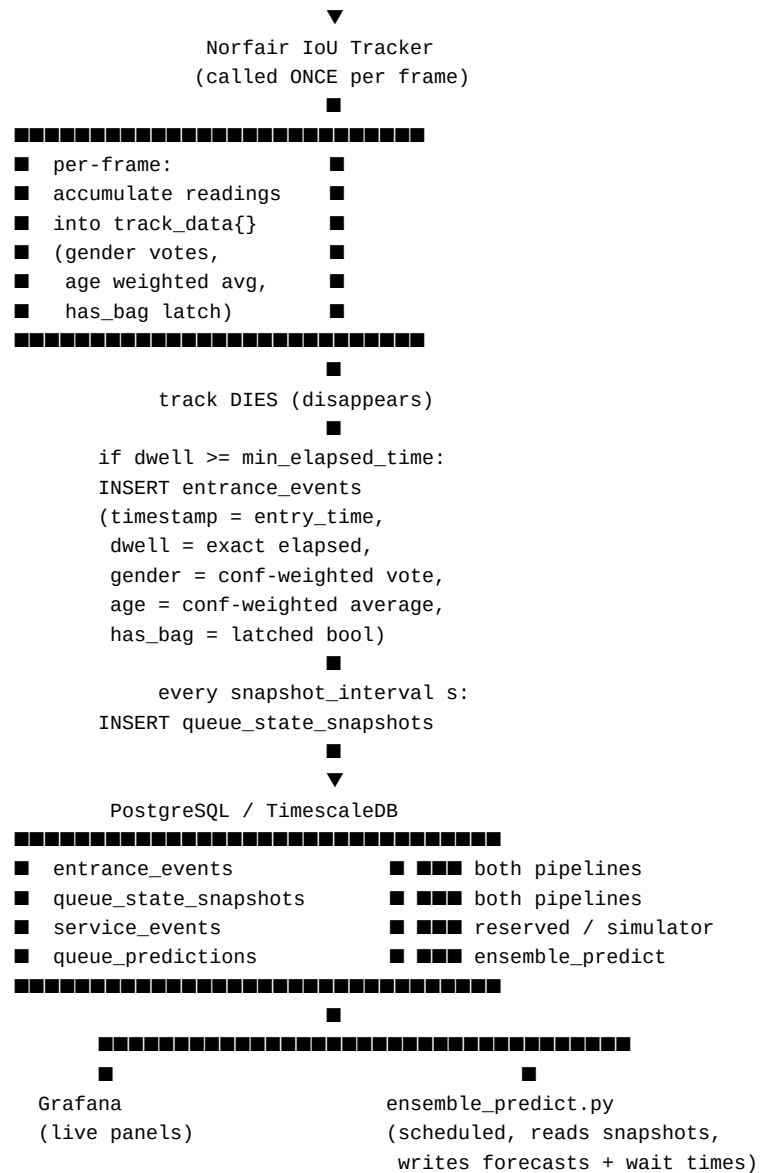
The forecasting layer reads the most recent snapshot to seed a real-time queue equation for wait-time estimation.

2. Repository Layout

Queue-Management/ ■■■■ Head-Detector/ ■ ■■■■ main.py ■ ■■■■ pick_zone.py ■ ■■■■ config.yml ■ ■■■■ config2.yml ■ ■■■■ utils/ ■ ■■■■ db_logger.py ■ ■■■■ queue_utils.py ■ ■■■■ yolo.py ■ ■■■■ Queue-Management-System-v2-main/ ■■■■ Queue-Management-System-v2-main/ ■■■■ queue_management_v2.py ■■■■ prophet_predict.py ■■■■ ensemble_predict.py ■■■■ config.yml / config2.yml ■■■■ models/ ■■■■ uniface/ ■■■■ utils/ ■■■■ db_logger.py ■■■■ queue_utils.py ■■■■ yolov9.py ■■■■ simulator/ ■ ■■■■ engine.py ■ ■■■■ entities.py ■ ■■■■ db.py ■ ■■■■ scenarios.py ■ ■■■■ calibrate.py ■ ■■■■ predict_viz.py ■ ■■■■ app.py ■ ■■■■ server.py ■ ■■■■ frontend/index.html ■ ■■■■ pygame_viewer.py ■ ■■■■ test_run.py ■■■■ run_simulator.py ■■■■ run_simulator.bat ■■■■ DEVELOPER_DOC.md ■■■■ SIMULATOR_OVERVIEW.md ■■■■ BUG_FIXES.md ■■■■ IQMS_FULL_DOC.md	 ← Pipeline A ← entry point ← ROI drawing tool ← camera + model settings ← tracker + debug settings ← PostgreSQL / TimescaleDB writer ← shared helpers, logging, IoU ← YOLOv9 ONNX wrapper ← Pipeline B entry point ← standalone Prophet forecast ← Prophet + LSTM + XGBoost forecast ← saved ML model files (auto-created) ← face detection / age-gender library ← discrete-event simulator core ← Person / Lane dataclasses ← simulator DB writer (SIM_* camera IDs) ← scenario presets ← calibrates from REAL DB data ← Prophet + Plotly for Streamlit prediction tab ← Streamlit dashboard ← async WebSocket engine server ← HTML5 Canvas 2D store-floor visualisation ← local Pygame animated viewer ← smoke test ← unified simulator launcher ← Windows wrapper ← this document
--	--

3. Architecture & Data Flow





4. Pipeline A — Head-Detector

Entry point: Head-Detector/main.py

4.1 Startup sequence

- Parse CLI args (--execution_provider, --inference_type, --source, --view-img)
- Load config.yml and config2.yml
- Initialise DBLogger → creates/migrates hypertables
- Load YOLOv9 ONNX model
- Initialise Norfair tracker
- Start VideoStream background thread
- Enter main detection loop

4.2 Per-frame loop

read frame

```

→ resize to 640×480
→ YOLOv9 inference → boxes[]
→ for each box:
    filter class_id == 0 (head)
    skip if obj.age == 0 (brand-new, unconfirmed track)
    check tip_point inside ROI polygon
    if inside → append to h_dets
← end for loop
→ build norfair_detections from h_dets (once per frame)
→ tracker.update(norfair_detections) (once per frame)
→ detect died tracks (prev_track_ids - current_track_ids)
→ for each DIED track_id:
    if dwell >= min_elapsed_time:
        INSERT entrance_events
        (timestamp = entry_dt, dwell = elapsed seconds)
→ for each tracked_object:
    accumulate face readings into track_data{}
    draw dwell label on frame
→ every snapshot_interval seconds:
    INSERT queue_state_snapshots

```

4.3 Tracker configuration (config2.yml)

Key	Default	Meaning
max_distance_between_points	6	IoU distance threshold
max_age	6	Frames to keep a track without detection
expect_fps	5	Used to convert obj . age to seconds
tip_offset	0.5	Fraction along box width for ROI tip point
min_elapsed_time	1	Minimum dwell (seconds) for a DB insert
debug_mode	true	Track IDs on overlay + debug snapshots
snapshot_interval	10	Seconds between snapshot writes

4.4 CLI arguments

Argument	Values	Default
--execution_provider	cpu, cuda, tensorrt	tensorrt
--inference_type	fp16, int8	fp16
--view_img	flag	off
--source	path or stream URL	config.yml RTSP

4.5 ROI tool

python pick_zone.py

Left-click adds points, Z undoes, Enter/Space prints the points: block to paste into config.yml.

5. Pipeline B — Queue-Management-System-v2

Entry point: queue_management_v2.py

5.1 Face analytics

Runs FaceAnalyzer (Uniface/RetinaFace) on the full frame after collecting ROI body detections. Faces are matched to bodies via containment IoU (`min_iou` from `config2.yml`). Every body detection in the ROI is passed to the tracker regardless of face match — occluded faces fall back to `gender='unknown'`, `age=None`.

5.2 Gender / age aggregation (confidence-weighted)

Readings are accumulated across the full track lifetime:

- **Gender:** confidence-weighted vote — each frame adds face confidence to the running gender total; winner at death is stored
- **Age:** confidence-weighted average across all valid frames

```
track_data[track_id] = {
    "gender_votes": {"male": 0.0, "female": 0.0, "unknown": 0.0},
    "age_sum": 0.0,
    "age_weight": 0.0,
    "best_conf": 0.0,
    "has_bag": False,
    "entry_dt": datetime.now(),
}
```

5.3 Bag detection (COCO classes 24 + 26)

COCO class	Object
24	Backpack
26	Handbag

Suitcases (class 28) are excluded. Each frame, bag bounding boxes are collected separately.

`_person_has_bag(p_box, bags)` expands the person box by 30% and checks whether any bag centre falls inside. `has_bag` is **latched** — once True it never resets for that track.

```
BAG_CLASSES = {24, 26}

def _person_has_bag(p_box, bags):
    px1, py1, px2, py2 = p_box
    pw = px2 - px1; ph = py2 - py1
    ex1, ey1 = px1 - pw*0.3, py1 - ph*0.3
    ex2, ey2 = px2 + pw*0.3, py2 + ph*0.3
    for bx1, by1, bx2, by2 in bags:
        bcx = (bx1+bx2)/2; bcy = (by1+by2)/2
        if ex1 <= bcx <= ex2 and ey1 <= bcy <= ey2:
            return True
    return False
```

Debug mode renders an orange **BAG** label on the bounding box when detected.

5.4 Insert-at-death pattern

Same as Pipeline A: one `entrance_events` row written at track death with `entry_time = entry_dt`. Includes `has_bag`, confidence-weighted gender, and confidence-weighted `age_estimate`.

5.5 Snapshot logging

Every `snapshot_interval` seconds, live queue metrics are written to `queue_state_snapshots`.

6. Forecasting Layer

6.1 prophet_predict.py — quick single-model forecast

Console-only. Reads entrance_events, pads with 7 days of Poisson-simulated history, trains Prophet, prints 60-minute forecast and snapshot-seeded wait estimates. Does **not** write to DB.

```
python prophet_predict.py --source REAL
python prophet_predict.py --source SIM
python prophet_predict.py --source ALL
```

6.2 ensemble_predict.py — production forecast

Trains Prophet + LSTM + XGBoost, writes results to queue_predictions. Recommended: schedule every 15–30 minutes.

```
python ensemble_predict.py --source REAL
```

Top-level tuning constants:

Constant	Default	Effect
W_PROPHET	0.40	Prophet share
W_LSTM	0.30	LSTM share
W_XGB	0.30	XGBoost share
SEQUENCE_LEN	12	LSTM lookback ($12 \times 5 \text{ min} = 60 \text{ min}$)
FORECAST_STEPS	12	Steps ahead (60 min)
MODEL_MAX_AGE_HOURS	24	Retrain threshold
MIN_REAL_DAYS_FOR_SIM	14	Skip sim history once real data is rich enough

LSTM architecture

```
Input → LSTM(64, return_sequences=True) → Dropout(0.2)
      → LSTM(32) → Dropout(0.2) → Dense(1)
```

Optimizer: adam, Loss: mse, Epochs: 20, Batch: 32. MinMax-scaled on real data only.

XGBoost features

Feature	Description
hour, minute_of_hour	Time of day
day_of_week, is_weekend	Weekday pattern
lag_1, lag_2, lag_3	5/10/15 min lags
lag_12	60 min lag (same hour last period)
rolling_mean_6	30-min trailing average

Ensemble combination

```
ensemble = 0.40 * prophet_vals + 0.30 * lstm_vals + 0.30 * xgb_vals
ensemble = ensemble.clip(min=0).round(1)
```

Dynamic wait-time estimation

```
# Seed from latest queue_state_snapshots row
current_queue = snap.queue_count
avg_dwell_min = snap.avg_dwell_sec / 60
service_per_bucket = active_lanes * (5.0 / avg_dwell_min)

# Roll backlog forward
for arrivals in ensemble_vals:
    running_queue = max(0, running_queue + arrivals - service_per_bucket)
    wait_min = (running_queue / service_per_bucket) * avg_dwell_min
```

wait_15m = step 3, wait_30m = step 6. Status: OK (<5 min), BUSY (5–10 min), ALERT (≥10 min).

7. Database Schema

Database: iqms (PostgreSQL + TimescaleDB). All tables created automatically on startup. All time columns use TIMESTAMPTZ. Migration blocks auto-convert legacy TIMESTAMP columns.

7.1 entrance_events (*hypertable*)

Column	Type	Notes
id	BIGSERIAL	auto-increment
timestamp	TIMESTAMPTZ NOT NULL DEFAULT NOW()	= actual entry time
track_id	INT	Norfair session ID (resets on restart)
gender	VARCHAR(20)	'unknown' in Pipeline A; 'male'/'female' in B
age_estimate	FLOAT	NULL in Pipeline A
has_bag	BOOLEAN DEFAULT FALSE	detected in Pipeline B; always FALSE in Pipeline A
has_caddy	BOOLEAN DEFAULT FALSE	simulator only
is_group	BOOLEAN DEFAULT FALSE	simulator only
group_id	VARCHAR(100)	simulator only
confidence	FLOAT	detection confidence
camera_id	VARCHAR(100)	from config.yml `camID`
dwell_seconds	FLOAT DEFAULT 0	total ROI dwell; written at track death

Insert pattern: one row per confirmed track, at death.

Camera naming: real cameras use plain names (Bosch_Camera_Entrance); simulated / test runs use SIM_ prefix.

7.2 queue_state_snapshots (*hypertable*)

Written every snapshot_interval seconds (default 10 s).

Column	Type	Notes
id	BIGSERIAL	
timestamp	TIMESTAMPTZ NOT NULL DEFAULT NOW()	
camera_id	VARCHAR(100)	
queue_count	INT	active tracked persons at snapshot time

Column	Type	Notes
avg_dwell_sec	FLOAT	mean dwell of current tracked persons
max_dwell_sec	FLOAT	max dwell of current tracked persons
active_lanes	INT DEFAULT 2	from `config.yml`

7.3 service_events (hypertable)

Reserved for future exit-ROI integration. Currently populated only by the simulator.

Column	Type	Notes
id	BIGSERIAL	
timestamp	TIMESTAMPTZ	
camera_id	VARCHAR(100)	
track_id	INT	
lane_id	INT	simulator: which checkout lane
total_dwell_sec	FLOAT	total time from entry to service end

7.4 queue_predictions (hypertable)

Written by ensemble_predict.py.

Column	Type	Notes
id	BIGSERIAL	
predicted_at	TIMESTAMPTZ	when the forecast run happened
prediction_for	TIMESTAMPTZ	the 5-min bucket being predicted
prophet_yhat	NUMERIC(8,2)	
lstm_yhat	NUMERIC(8,2)	
xgb_yhat	NUMERIC(8,2)	
ensemble_yhat	NUMERIC(8,2)	weighted average
est_wait_minutes	NUMERIC(8,2)	queue-model wait at this bucket
wait_15m	NUMERIC(8,2)	wait at +15 min horizon
wait_30m	NUMERIC(8,2)	wait at +30 min horizon
status	VARCHAR(10)	OK / BUSY / ALERT

7.5 Recommended Grafana queries

Live entry rate (3-min buckets):

```
SELECT time_bucket('3 minutes', timestamp) AS time, COUNT(*) AS entries
FROM entrance_events WHERE $__timeFilter(timestamp)
GROUP BY time ORDER BY time;
```

Live queue size:

```
SELECT timestamp AS time, queue_count, avg_dwell_sec, active_lanes
FROM queue_state_snapshots WHERE $__timeFilter(timestamp) ORDER BY time;
```

Forecast overlay:

```
-- Actual
SELECT time_bucket('3 minutes', timestamp) AS time, COUNT(*) AS actual
```



```

FROM entrance_events WHERE $__timeFilter(timestamp)
GROUP BY time ORDER BY time;

-- Forecast
SELECT prediction_for AS time, ensemble_yhat, est_wait_minutes
FROM queue_predictions WHERE prediction_for BETWEEN NOW() AND NOW() + INTERVAL '1 hour'
ORDER BY prediction_for;

```

Wait horizon panel:

```

SELECT predicted_at AS time, wait_15m, wait_30m
FROM queue_predictions WHERE $__timeFilter(predicted_at) ORDER BY predicted_at;

```

8. Configuration Reference

`config.yml`

Key	Example	Description
camID	Bosch_Camera_Entrance	Stored in DB `camera_id`
ip_address	192.168.1.48:554/?inst=3	RTSP host:port/path
username / password	—	RTSP auth
custom_model	models/yolov9_s.onnx	ONNX model path
score	0.3	Object confidence threshold
active_lanes	2	Open checkout lanes
points	[[x,y], ...]	ROI polygon vertices

`config2.yml`

Key	Default	Description
debug_mode	true	Overlay track IDs; save debug snapshots
expect_fps	5	Expected processing FPS
max_age	6	Frames before a lost track is dropped
max_distance_between_points	6	IoU distance threshold
tip_offset	0.5	ROI tip-point horizontal offset (0–1)
min_iou	0.01	Minimum face-body containment (Pipeline B only)
snapshot_interval	10	Seconds between snapshot writes
min_elapsed_time	1	Minimum dwell (s) for a DB insert

9. Grafana / TimescaleDB Integration

Prerequisites

- PostgreSQL 14+ with TimescaleDB 2.x
- Grafana 10+ with built-in PostgreSQL datasource

Datasource settings

Host: localhost:5432
Database: iqms
User: postgres
Password: 0000
TimescaleDB: ☒ enabled

Why TIMESTAMPTZ is required

Grafana time series panels detect the time field by data type. `TIMESTAMP WITHOUT TIME ZONE` is not recognised. All tables use `TIMESTAMPTZ NOT NULL DEFAULT NOW()`.

Recommended panels

Panel	Type	Metric
People in queue now	Stat	queue_count` from latest snapshot
Queue over time	Time series	queue_count` from snapshots
Entry rate	Time series	3-min bucket count from `entrance_events
Avg dwell	Time series	avg_dwell_sec` from snapshots
Forecast vs actual	Time series	overlay entrance_events + `queue_predictions
Wait horizons	Time series	wait_15m, wait_30m` from predictions
Wait status	State timeline	status` from predictions
Gender split	Pie chart	GROUP BY gender on entrance_events
Bag rate	Stat	COUNT(*) FILTER (WHERE has_bag) on entrance_events

10. Setup & First Run

10.1 PostgreSQL + TimescaleDB

```
CREATE DATABASE iqms;  
\c iqms  
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;
```

All tables are created automatically on first pipeline startup.

10.2 Python environment

```
pip install psycpg2-binary pandas numpy prophet xgboost tensorflow \  
scikit-learn opencv-python norfair shapely onnxruntime pyyaml
```

For GPU:

```
pip install onnxruntime-gpu
```

10.3 Head-Detector first run

```
python pick_zone.py          # define ROI (once)  
python main.py --execution_provider cpu --view-img
```

10.4 Pipeline B first run

```
python queue_management_v2.py --device cpu --view_img True
```

10.5 Forecasting

```
python prophet_predict.py
python ensemble_predict.py --source REAL
```

10.6 Verify DB

```
SELECT COUNT(*), MAX(timestamp) FROM entrance_events;
SELECT * FROM queue_state_snapshots ORDER BY timestamp DESC LIMIT 5;
SELECT prediction_for, ensemble_yhat, wait_15m, wait_30m, status
FROM queue_predictions ORDER BY prediction_for DESC LIMIT 12;
```

11. Simulator

Full details in SIMULATOR_OVERVIEW.md. Key points:

11.1 Three-service architecture

Service	Port	Purpose
Streamlit app	8501	Controls, prediction tab
WebSocket engine	8080	server.py` – runs simulation, broadcasts 30x/s
HTTP static server	8081	Canvas visualisation

11.2 Event model

Event	Trigger	Result
ARRIVAL	Exponential inter-arrival gap	Creates Person, schedules `ENTRANCE_DONE
ENTRANCE_DONE	After bakery / caddy dwell	Moves to shop floor
SHOPPING_DONE	After shop time	Joins shortest queue
SERVICE_START	Lane free	Begins checkout
SERVICE_END	After service duration	Logs to DB
SNAPSHOT	Every `snapshot_interval_seconds	Writes queue snapshot

11.3 Person characteristics

Field	Description
gender	male / female
age	float 18–80
has_bag	small basket → slightly shorter shop time
has_caddy	trolley → +30–90s entrance + +12–28 min shop

Field	Description
is_group	+10–25s entrance + +5–15 min shop
picks_bakery	+20–90s entrance + +2–6 min shop
is_express	~25% of non-caddy customers; 2–5 min shop time

11.4 Shop-time model

Base time is time-of-day aware:

Time period	Base shop time
is_express	2–5 min
Lunchtime (12–14 h)	4–10 min
Morning (8–10 h)	8–18 min
After-work (17–19 h)	10–20 min
Other	5–15 min

Modifiers add on top: caddy (+12–28 min), bag without caddy (–6—2 min), group (+5–15 min), group+caddy (+5–10 min), bakery (+2–6 min), age>65 (+5–12 min), age<25 (–4–0 min).

11.5 Launcher commands

```
python run_simulator.py dashboard
python run_simulator.py batch normal_day
python run_simulator.py calibrate --days 30
python run_simulator.py viewer normal_day
python run_simulator.py test
```

11.6 Canvas emoji legend

Badge	Meaning
■	Express shopper
■	Has caddy
■	Picks bakery
■	Has bag

Ring colour	Zone
Green	Entrance
Blue / Pink	Shopping floor
Amber	Waiting in queue
Red	Being served

12. Bug Fixes & Data Quality History

Original bugs fixed (pre-April 2026)

#	File(s)	Description
1	Head-Detector/main.py	Tracker called N times per frame → duplicate inserts
2	Both `db_logger.py	Table mismatch queue_events vs `entrance_events
3	Both pipelines	update_dwell` firing 3× per 2-second window
4	All tables	TIMESTAMP` blocked Grafana + TimescaleDB
5	Both forecast scripts	drop_duplicates` kept SIM rows over REAL
6	Both forecast scripts	Mixed granularity (1-min real vs 5-min sim)
7	Both forecast scripts	Hardcoded -2 h timezone offset
8	queue_management_v2.py	Body-only detections dropped → undercounting
9	ensemble_predict.py	Fixed service-rate wait formula
10	ensemble_predict.py	LSTM scaler fitted on SIM data distorted real scale
11	ensemble_predict.py	Repeated model retraining every run
S1-S7	simulator/	Various simulator-specific fixes (WebSocket, iframe, Prophet, fragments)

April 2026 — Data quality & track lifecycle

Fix 12 — Tracker re-detection spam (insert-at-death)

Brief occlusions caused Norfair to drop and re-create tracks, each triggering an immediate insert. Peak observed: 644 entries in one 3-minute bucket (normal: 10–30).

Track lifecycle redesigned:

- Accumulate readings in `track_data{}` while alive
- Insert **once at track death** with `entry_time = entry_dt` (first-frame timestamp)
- Discard tracks with `dwell < min_elapsed_time`

Fix 13 — Confidence-weighted gender/age

Gender and age now represent the consensus across all frames where a face was detected, not just the first frame.

Fix 14 — Test-mode camera_id contaminating real data

`camID = 'SIM_live_test_video'` in both pipelines when running against test video.

Fix 15 — Historical DB cleanup (2026-04-12)

```
-- Restart duplicates: kept earliest row per (track_id, camera_id, day)
DELETE FROM entrance_events WHERE ... id NOT IN (SELECT MIN(id) ...);
-- Deleted 8610 rows

-- Brief re-detections (NULL dwell, never updated)
DELETE FROM entrance_events WHERE ... AND dwell_seconds IS NULL;
-- Deleted 10791 rows
```

Fix 16 — Camera rename

Old	New
Bosch_Camera_169	Bosch_Camera_Entrance
Bourgogne_Sortie_Gauche	Bosch_Camera_Exit

Applied to all rows in entrance_events and queue_state_snapshots (2026-04-12).

Fix 17 — Forecast bucket: 5 min → 3 min

All `time_bucket('5 minutes', ...)` → `time_bucket('3 minutes', ...)` in `predict_viz.py` and `app.py`. Gives finer peak resolution while remaining stable with the current data volume.

Fix 18 — Bucket enriched with gender, age, dwell

Each 3-minute bucket now also carries:

```

COUNT(*) FILTER (WHERE gender = 'male')           AS male_count,
COUNT(*) FILTER (WHERE gender = 'female')         AS female_count,
ROUND(AVG(age_estimate)::numeric, 1)               AS avg_age,
ROUND(AVG(dwell_seconds) FILTER (WHERE dwell_seconds > 0)::numeric, 1) AS avg_dwell_sec

```

Fix 19 — Bag detection in Pipeline B (COCO 24 + 26)

YOLOv9 now also detects backpacks (class 24) and handbags (class 26) in the same inference pass. Bags are associated to persons via a 30%-expanded proximity check. `has_bag` is latched and written to `entrance_events` at track death.

Pipeline A always stores `has_bag=FALSE`.

13. Known Limitations

- `track_id` resets on app restart — use `timestamp` for cross-session analysis
- `service_events` not yet populated by live production pipelines (only simulator)
- `active_lanes` is static per session; restart required after config change
- `ensemble_predict.py` is a one-shot script, must be scheduled externally
- Gender/age bucket enrichment meaningful only for Pipeline B (Bosch_Camera_Entrance)
- Pipeline A always stores `has_bag=FALSE`, `gender='unknown'`, `age=NULL`
- Single-polygon ROI per pipeline instance
- Simulator writes to shared production tables under `SIM_*` prefix
- LSTM step-12 forecast carries 12 levels of compounded prediction error

Generated: 2026-04-12 | IQMS v2 with insert-at-death, bag detection, 3-min buckets